

Introduction à l'Analyse Asymptotique

et Application au calcul de complexités

12 janvier 2026

Table des matières

1	Introduction à l'Analyse Asymptotique	2
2	Notion de Négligeabilité	2
3	Les Trois Notations Fondamentales, notations de Landau	3
3.1	Grand O (O) : La Borne Supérieure (Pire Cas)	
3.2	Grand Omega (Ω) : La Borne Inférieure (Meilleur Cas)	
3.3	Grand Theta (Θ) : La Borne Serrée (L'Encadrement)	
4	Relation entre valeur et taille de l'entrée	4
5	Calcul de Complexité	4
5.1	Règles de simplification	
5.2	Algèbre des notations	
6	Classes de Complexité Usuelles	5
7	Étude de Cas : Quelques algorithmes classiques	6
7.1	Algorithmes Itératifs Simples	
8	Annexe : Implémentation Python	11

parindent 0ex

1 Introduction à l'Analyse Asymptotique

En informatique, comparer l'efficacité des algorithmes en mesurant le temps en secondes est peu fiable car cela dépend de la machine, du langage et de la charge système.

L'**analyse asymptotique** est une méthode mathématique qui évalue la performance d'un algorithme en fonction de la **taille de son entrée** (notée n), lorsque cette taille tend vers l'infini. On cherche l'**ordre de grandeur** de la croissance du nombre d'opérations (et donc du temps de calcul).

Il est important de noter que l'on peut analyser l'occupation mémoire (complexité spatiale) de la même manière. Cependant, **dans ce document, nous nous limiterons à l'analyse en temps** (complexité temporelle), exprimée comme une fonction du nombre d'opérations élémentaires.

2 Notion de Négligeabilité

Avant de formaliser les notations complexes, il est crucial de comprendre intuitivement ce que signifie qu'une quantité devient insignifiante par rapport à une autre lorsque n grandit. Ici on ne s'intéressera qu'à l'analyse au voisinage de l'infini (on peut faire la même étude au voisinage de 0 ou d'un point quelconque. On peut définir la négligeabilité au voisinage d'un point quelconque).

Définition 1 (Négligeabilité). On dit qu'une fonction $f(n)$ est **négligeable** devant $g(n)$ (noté $f(n) \ll g(n)$ ou $f(n) = o(g(n))$) si le rapport tend vers 0 :

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

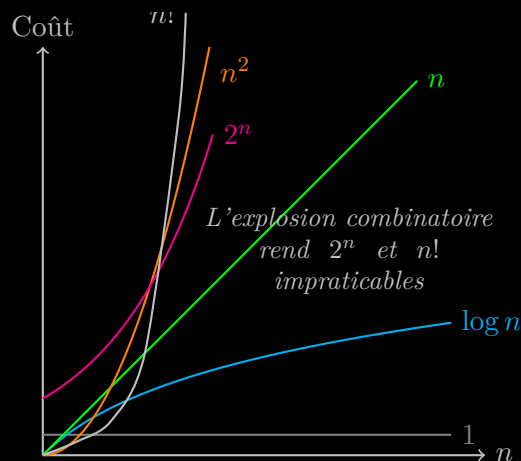
Cela signifie que pour des valeurs de n très grandes, $f(n)$ devient "invisible" si on regarde l'échelle de $g(n)$.

Exemple 1. — n est négligeable devant n^2 car $\lim_{n \rightarrow \infty} \frac{n}{n^2} = \lim_{n \rightarrow \infty} \frac{1}{n} = 0$.

— Une constante est négligeable devant $\log n$.

— Tout polynôme est négligeable devant une exponentielle ($n^{100} \ll 2^n$).

C'est ce concept qui nous autorise à simplifier les expressions de complexité. Si un algorithme "coûte" $T(n) = n^2 + n$ opérations, comme $n \ll n^2$, on retiendra uniquement que l'algorithme est quadratique (n^2).



Exercice 1 (Classement d'ordres de grandeur). Classez les fonctions suivantes par ordre de négligeabilité croissante (c'est-à-dire du plus efficace/rapide au plus coûteux/lent pour n très grand) :

$$n^3, \quad \sqrt{n}, \quad n!, \quad \log(n), \quad 2^n, \quad n \log(n), \quad 100, \quad n, \quad n \log(\log n), \quad e^n, \quad n \ln(\sqrt{n})$$

3 Les Trois Notations Fondamentales, notations de Landau

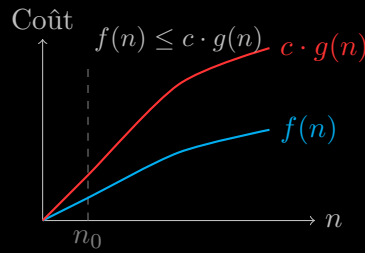
Soit $f(n)$ la fonction de coût de l'algorithme (ex : nombre d'opérations) et $g(n)$ une fonction de référence connue (ex : n , n^2).

3.1 Grand O (O) : La Borne Supérieure (Pire Cas)

L'algorithme ne sera jamais plus lent que cela. C'est un **plafond**.

Définition 2 (Grand O). $f(n) \in O(g(n))$ s'il existe une constante $c > 0$ et un rang n_0 tels que :

$$\forall n \geq n_0, \quad f(n) \leq c \cdot g(n)$$



3.2 Grand Omega (Ω) : La Borne Inférieure (Meilleur Cas)

L'algorithme prendra au moins ce temps-là. C'est un **plancher**.

Définition 3 (Grand Omega). $f(n) \in \Omega(g(n))$ s'il existe une constante $c > 0$ et un rang n_0 tels que :

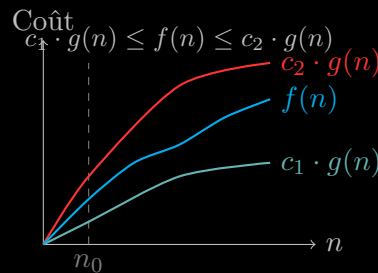
$$\forall n \geq n_0, \quad f(n) \geq c \cdot g(n)$$

3.3 Grand Theta (Θ) : La Borne Serrée (L'Encadrement)

L'algorithme se comporte exactement comme $g(n)$ à une constante près. Le nombre d'opérations (temps de calcul) est **borné**.

Définition 4 (Grand Theta). $f(n) \in \Theta(g(n))$ s'il existe des constantes $c_1, c_2 > 0$ et un rang n_0 tels que :

$$\forall n \geq n_0, \quad c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$



4 Relation entre valeur et taille de l'entrée

Il est important de distinguer la **valeur** d'un nombre a et sa **taille** en mémoire (nombre de bits).

Proposition 1. Soit a un entier strictement positif et n sa longueur en bits (nombre de chiffres binaires). Alors :

$$\log_2(a) = \Theta(n)$$

Démonstration. Si a s'écrit sur n bits, cela signifie que le bit de poids fort (2^{n-1}) est à 1. On a donc l'encadrement suivant pour la valeur de a :

$$2^{n-1} \leq a < 2^n$$

En appliquant le logarithme en base 2 (fonction strictement croissante) :

$$\log_2(2^{n-1}) \leq \log_2(a) < \log_2(2^n)$$

$$n - 1 \leq \log_2(a) < n$$

Pour tout $n \geq 2$, on a $n - 1 \geq \frac{n}{2}$. On peut donc écrire :

$$\frac{1}{2}n \leq \log_2(a) \leq 1 \cdot n$$

Il existe donc des constantes $c_1 = 0.5$, $c_2 = 1$ et $n_0 = 2$ telles que $\log_2(a)$ soit compris entre $c_1 n$ et $c_2 n$. Par définition, $\log_2(a) = \Theta(n)$. $\square$

Conséquence : Lorsqu'on dit que la complexité de la multiplication russe ou de l'exponentiation rapide est en $\mathcal{O}(\log a)$, cela équivaut à dire qu'elle est **linéaire** par rapport au nombre de bits de a ($\mathcal{O}(n)$).

5 Calcul de Complexité

5.1 Règles de simplification

Pour trouver la classe de complexité, on applique deux règles simples :

1. **Garder le terme dominant :** On ne garde que le terme qui croît le plus vite quand $n \rightarrow \infty$.

$$n^2 + 100n + 5 \xrightarrow{\text{simplification}} n^2$$

2. **Ignorer les constantes :** Les facteurs multiplicatifs ne changent pas la classe.

$$3n^2 \xrightarrow{\text{simplification}} \mathcal{O}(n^2)$$

5.2 Algèbre des notations

- **Séquence (Addition) :** On prend le maximum.

$$\mathcal{O}(f(n)) + \mathcal{O}(g(n)) = \mathcal{O}(\max(f(n), g(n)))$$

Exemple : Une boucle $\mathcal{O}(n)$ suivie d'une boucle imbriquée $\mathcal{O}(n^2)$ donne $\mathcal{O}(n^2)$.

- **Imbrication (Multiplication) :** On multiplie les complexités.

$$\mathcal{O}(f(n)) \times \mathcal{O}(g(n)) = \mathcal{O}(f(n) \cdot g(n))$$

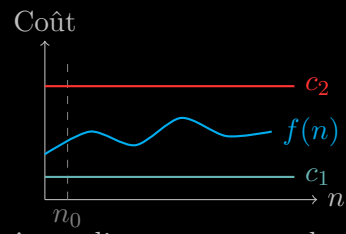
Exemple : Une boucle $\mathcal{O}(n)$ contenant un appel de fonction $\mathcal{O}(\log n)$ donne $\mathcal{O}(n \log n)$.

6 Classes de Complexité Usuelles

Voici les classes classées de la plus efficace à la moins efficace :

Notation	Nom	Exemple typique
$\mathcal{O}(1)$	Constante	Accès à un élément de tableau / Hash map
$\mathcal{O}(\log n)$	Logarithmique	Recherche binaire (Dichotomie)
$\mathcal{O}(n)$	Linéaire	Parcours simple de tableau
$\mathcal{O}(n \log n)$	Linéarithmique	Tri rapide (QuickSort), Tri fusion
$\mathcal{O}(n^2)$	Quadratique	Boucles imbriquées (Tri à bulles)
$\mathcal{O}(2^n)$	Exponentielle	Récursion naïve (Fibonacci), Voyageur de commerce

Illustration de la complexité constante $\Theta(1)$ Une fonction est en $\Theta(1)$ si son temps d'exécution reste borné par deux constantes, quelle que soit la taille de l'entrée n .



Le coût ne diverge pas quand $n \rightarrow \infty$

7 Étude de Cas : Quelques algorithmes classiques

Nous appliquons ici l'analyse asymptotique à plusieurs problèmes classiques, en commençant par des structures simples.

Calculer la complexité des algorithmes suivant. On précisera la complexité de chaque ligne et le calcul final.

7.1 Algorithmes Itératifs Simples

Somme des premiers entiers

Algorithme 1 Somme des n premiers entiers

Entrée : $n \in \mathbb{N}$

Sortie : Somme $S = \sum_{i=1}^n i$

```
1: procedure SOMME( $n$ )  
2:    $S \leftarrow 0$   
3:   for  $i \leftarrow 1$  to  $n$  do  
4:      $S \leftarrow S + i$   
5:   end for  
6:   return  $S$   
7: end procedure
```

Complexité Totale :

Maximum dans un tableau

Algorithme 2 Recherche du Maximum dans un tableau

Entrée : T un tableau de taille n

Sortie : La valeur maximale du tableau

```
1: procedure RECHERCHEMAX( $T, n$ )  
2:    $max \leftarrow T[1]$   
3:   for  $i \leftarrow 2$  to  $n$  do  
4:     if  $T[i] > max$  then  
5:        $max \leftarrow T[i]$   
6:     end if  
7:   end for  
8:   return  $max$   
9: end procedure
```

Complexité Totale :

Recherche de paires

Algorithme 3 Affichage de Paires (Boucles Imbriquées)

Entrée : $n \in \mathbb{N}$

Sortie : Affiche toutes les paires (i, j) possibles

```
1: procedure AFFICHEPAIRES( $n$ )  
2:   for  $i \leftarrow 1$  to  $n$  do  
3:     for  $j \leftarrow 1$  to  $n$  do  
4:       Afficher  $(i, j)$   
5:     end for  
6:   end for  
7: end procedure
```

Complexité Totale :

Exponentiation Naïve : consiste à multiplier la base a par elle-même n fois

Algorithme 4 Exponentiation Naïve

Entrée : $a \in \mathbb{R}$ (la base), $n \in \mathbb{N}$ (l'exposant)

Sortie : Résultat $r = a^n$

```
1: procedure EXPNAIVE( $a, n$ )  
2:    $r \leftarrow 1$   
3:   for  $i \leftarrow 1$  to  $n$  do  
4:      $r \leftarrow r \times a$   
5:   end for  
6:   return  $r$   
7: end procedure
```

Complexité Totale :

Exponentiation Rapide itérative

Cette méthode utilise la représentation binaire de l'exposant pour réduire drastiquement le nombre de multiplications. C'est l'algorithme standard utilisé en cryptographie (RSA).

Algorithme 5 Exponentiation Rapide (Itérative)

Entrée : $a \in \mathbb{R}, n \in \mathbb{N}$

Sortie : Résultat $r = a^n$

```
1: procedure EXPRAPIDE( $a, n$ )
2:    $r \leftarrow 1$ 
3:    $base \leftarrow a$ 
4:    $exposant \leftarrow n$ 
5:   while  $exposant > 0$  do
6:     if  $exposant \pmod{2} = 1$  then
7:        $r \leftarrow r \times base$ 
8:     end if
9:      $base \leftarrow base \times base$ 
10:     $exposant \leftarrow \lfloor exposant/2 \rfloor$ 
11:  end while
12:  return  $r$ 
13: end procedure
```

Complexité Totale :

Copie chaque élément d'une matrice carrée A vers une matrice B .

Algorithme 6 Copie de Matrice Carrée

Entrée : A une matrice $n \times n$

Sortie : Matrice B telle que $B = A$

```
1: procedure COPIEMATRICE( $A, n$ )
2:   for  $i \leftarrow 1$  to  $n$  do
3:     for  $j \leftarrow 1$  to  $n$  do
4:        $B[i][j] \leftarrow A[i][j]$ 
5:     end for
6:   end for
7:   return  $B$ 
8: end procedure
```

Complexité Totale :

Multiplication matricielle naïve

Algorithme 7 Multiplication Matricielle naïve

Entrée : A, B deux matrices $n \times n$

Sortie : Matrice $C = A \times B$

```
1: procedure MULTMATRICE( $A, B, n$ )
2:   for  $i \leftarrow 1$  to  $n$  do
3:     for  $j \leftarrow 1$  to  $n$  do
4:        $Somme \leftarrow 0$ 
5:       for  $k \leftarrow 1$  to  $n$  do
6:          $Somme \leftarrow Somme + A[i][k] \times B[k][j]$ 
7:       end for
8:        $C[i][j] \leftarrow Somme$ 
9:     end for
10:  end for
11:  return  $C$ 
12: end procedure
```

Complexité Totale :

Multiplication Russe (Paysanne)

Cette méthode ancienne permet de multiplier deux entiers a et b en n'utilisant que des multiplications par 2, des divisions par 2 et des additions. Elle est structurellement très proche de l'exponentiation rapide.

Algorithme 8 Multiplication Russe

Entrée : $a, b \in \mathbb{N}$ (les deux facteurs)

Sortie : Produit $p = a \times b$

```
1: procedure MULTRUSSE( $a, b$ )
2:    $p \leftarrow 0$ 
3:   while  $a > 0$  do
4:     if  $a \bmod 2 = 1$  then
5:        $p \leftarrow p + b$ 
6:     end if
7:      $b \leftarrow b \times 2$ 
8:      $a \leftarrow \lfloor a/2 \rfloor$ 
9:   end while
10:  return  $p$ 
11: end procedure
```

Complexité Totale :

Tri à bulles (Bubble sort)

Le tri à bulles est un algorithme de tri simple qui consiste à faire remonter progressivement les plus grands éléments vers la fin du tableau (comme des bulles).

Algorithme 9 Tri à Bulles

Entrée : T un tableau de n éléments

Sortie : Le tableau T trié par ordre croissant

```
1: procedure TRIBULLES( $T, n$ )
2:   for  $i \leftarrow 1$  to  $n$  do
3:     for  $j \leftarrow 1$  to  $n - i$  do
4:       if  $T[j] > T[j + 1]$  then
5:         Echanger  $T[j]$  et  $T[j + 1]$ 
6:       end if
7:     end for
8:   end for
9:   return  $T$ 
10: end procedure
```

Complexité Totale :

8 Annexe : Implémentation Python

Mettre en oeuvre, en python, les algorithmes précédents et vérifier empiriquement les complexités calculées précédemment. On utilisera pour cela les modules `time`, `random`, et `matplotlib`. Pour chaque algorithme on tracera le temps de calcul en fonction de n (taille des entrées).

```
1 import time
2 import random
3 import matplotlib.pyplot as plt
4 import sys
5
6 # Augmenter la limite de recursion si necessaire (pas utile ici car
  versions iteratives)
7 sys.setrecursionlimit(2000)
8
9
10 def exp_naive(a, n, mod=None):
11     res = 1
12
13     return res
14
15 def exp_rapide(a, n, mod=None):
16     res = 1
17     base = a
18
19     return res
20
21 def mult_russe(a, b):
22     res = 0
23
24     return res
25
26
27 def generer_matrice(n):
28     return [[random.randint(1, 10) for _ in range(n)] for _ in range(n)]
29
30 def copie_matrice(A):
31
32     return B
33
34 def mult_matrice(A, B):
35
36     return C
37
38
39 def bubble_sort(arr):
40
41     return T
42
43 # --- OUTILS DE MESURE ---
44
45 def mesurer_temps(fonction, *args):
46     start = time.perf_counter()
47     fonction(*args)
48     return time.perf_counter() - start
49
50 def benchmark_algos():
51     print("Demarrage du benchmark... (cela peut prendre quelques
```

```

    secondes)")
52
53 # Donnees pour les graphiques
54 sizes_exp = [1000, 5000, 10000, 50000, 100000, 500000] # Tailles
    pour  $O(n)$  vs  $O(\log n)$ 
55 sizes_matrix = [10, 30, 50, 80, 100, 120] # Tailles pour  $O(n^2)$  vs  $O$ 
    ( $n^3$ )
56
57 res_exp = {'naive': [], 'rapide': [], 'russe': [], 'sizes':
    sizes_exp}
58 res_mat = {'copie': [], 'mult': [], 'bubble': [], 'sizes':
    sizes_matrix}
59
60 # 1. Benchmark Exponentiation & Russe
61 print("Test: Exponentiation & Mult Russe...")
62 a = 12345
63 for n in sizes_exp:
64     # On utilise un modulo pour que le cout de la multiplication
        reste constant  $O(1)$ 
65     # sinon Python gere des grands nombres et le cout de la
        multiplication augmente.
66     mod = 1000000007
67
68     t_naive = mesurer_temps(exp_naive, a, n, mod)
69     t_rapide = mesurer_temps(exp_rapide, a, n, mod)
70     t_russe = mesurer_temps(mult_russe, n, a) # n est le facteur
        decompose
71
72     res_exp['naive'].append(t_naive)
73     res_exp['rapide'].append(t_rapide)
74     res_exp['russe'].append(t_russe)
75
76 # 2. Benchmark Matrices & Tri
77 print("Test: Matrices & Tri...")
78 for n in sizes_matrix:
79     A = generer_matrice(n)
80     B = generer_matrice(n)
81     tab_tri = [random.randint(0, 1000) for _ in range(n*10)] # n*10
        pour avoir assez de matiere pour le tri
82
83     t_copie = mesurer_temps(copie_matrice, A)
84     t_mult = mesurer_temps(mult_matrice, A, B)
85     # Pour le tri, on utilise une taille relative a n pour voir l'
        effet quadratique
86     # Attention: pour comparer matrix mult ( $n^3$ ) et tri ( $k^2$ ), les
        echelles sont differentes
87     # Ici on compare juste matrix copy ( $n^2$ ) et matrix mult ( $n^3$ )
        principalement.
88
89     res_mat['copie'].append(t_copie)
90     res_mat['mult'].append(t_mult)
91
92 # 3. Benchmark Specifique Tri (Bubble) avec plus de donnees
93 sizes_sort = [100, 500, 1000, 1500, 2000]
94 res_sort = {'bubble': [], 'sizes': sizes_sort}
95 print("Test: Tri a bulles...")
96 for n in sizes_sort:
97     arr = [random.randint(0, 10000) for _ in range(n)]

```

```

98         t_bubble = mesurer_temps(bubble_sort, arr)
99         res_sort['bubble'].append(t_bubble)
100
101     return res_exp, res_mat, res_sort
102
103 def plot_results(res_exp, res_mat, res_sort):
104     fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(18, 5))
105
106     # Graphique 1 : Lineaire vs Logarithmique
107     ax1.plot(res_exp['sizes'], res_exp['naive'], 'r-o', label='Exp Naive
108             0(n)')
109     ax1.plot(res_exp['sizes'], res_exp['rapide'], 'g-o', label='Exp
110             Rapide 0(log n)')
111     ax1.plot(res_exp['sizes'], res_exp['russe'], 'b--o', label='Mult
112             Russe 0(log n)')
113     ax1.set_title('Lineaire vs Logarithmique')
114     ax1.set_xlabel('Taille n')
115     ax1.set_ylabel('Temps (s)')
116     ax1.legend()
117     ax1.grid(True)
118
119     # Graphique 2 : Quadratique vs Cubique (Matrices)
120     ax2.plot(res_mat['sizes'], res_mat['copie'], 'b-o', label='Copie
121             Matrice 0(n^2)')
122     ax2.plot(res_mat['sizes'], res_mat['mult'], 'r-o', label='Mult
123             Matrice 0(n^3)')
124     ax2.set_title('Operations Matricielles')
125     ax2.set_xlabel('Taille matrice n x n')
126     ax2.set_ylabel('Temps (s)')
127     ax2.legend()
128     ax2.grid(True)
129
130     # Graphique 3 : Tri Quadratique
131     ax3.plot(res_sort['sizes'], res_sort['bubble'], 'orange', marker='o',
132             label='Bubble Sort 0(n^2)')
133     ax3.set_title('Tri a Bulles')
134     ax3.set_xlabel('Nombre d\'elements n')
135     ax3.set_ylabel('Temps (s)')
136     ax3.legend()
137     ax3.grid(True)
138
139     plt.tight_layout()
140     plt.show()
141
142 if __name__ == "__main__":
143     results = benchmark_algos()
144     plot_results(*results)

```